

Laboratório 6

Manipulação de Arquivos

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz
marcosleipnitz@ic.ufrj.br

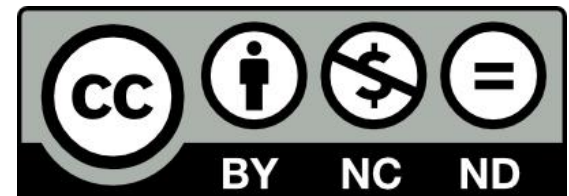


UFRJ

UNIVERSIDADE FEDERAL
DO RIO DE JANEIRO



INSTITUTO DE
COMPUTAÇÃO
UFRJ



Exercício 1

Faça um programa que recebe como entrada o nome de um arquivo texto. Seu programa deve calcular e exibir a quantidade de vezes que cada uma das vogais aparecem no arquivo (deve ser case insensitive e não precisa incluir vogais acentuadas na contagem).

Exemplo de arquivo de entrada:

Este eh um exemplo de arquivo de entrada. O importante eh contar as vogais caso uma vogal nao apareca, exibir zero

Saída na tela (stdout):

a 14
e 13
i 5
o 10
u 3

Exercício 2

Faça um programa que recebe como entrada o nome de um arquivo texto e o nome de um arquivo de saída. Esses dados devem vir da chamada do programa (via linha de comando). No arquivo de saída, deverá ser escrito o arquivo de entrada com os caracteres em maiúsculo. veja o exemplo a seguir:

Arquivo de entrada:

```
Hoje eh dia de prova de Prog 2  
A prova esta facil  
Vou tirar uma boa nota
```

Arquivo de saída:

```
HOJE EH DIA DE PROVA DE PROG 2  
A PROVA ESTA FACIL  
VOU TIRAR UMA BOA NOTA
```

Exercício 3

Faça um programa que recebe como entrada o nome de um arquivo texto e o nome de um arquivo de saída. Esses dados devem vir da chamada do programa (via linha de comando). Cada linha do arquivo de entrada possui até 200 caracteres. No arquivo de saída, deverá ser escrito o arquivo de entrada com o conteúdo de cada linha na ordem inversa. Veja um exemplo:

Arquivo de entrada:

```
Hoje eh dia de prova de Prog 2  
A prova esta facil  
Vou tirar uma boa nota
```

Arquivo de saída:

```
2 gorP ed avorp ed aid he ejoH  
licaf atse avorp A  
aton aob amu rarit uoV
```

Exercício 4

Os alunos de uma turma de Programação de Computadores II fizeram três avaliações: duas provas e um trabalho. Escreva um programa que leia do teclado o nome e as três notas de cada aluno dessa turma nas avaliações e os **grave em um arquivo texto** na seguinte ordem: nome do aluno, nota da primeira prova (P1), nota da segunda prova (P2) e nota do trabalho (T). O arquivo deverá armazenar os dados de cada aluno em uma linha do arquivo. O número de alunos da turma, bem como o nome do arquivo de saída, deverão ser solicitados ao usuário. Em seguida, abra o arquivo criado usando um editor de textos qualquer e verifique o resultado do programa.

Exercício 5

Escreva um programa que leia do arquivo gerado no exercício anterior (os nomes e as três notas dos alunos). O seu programa deve calcular a média de cada aluno considerando a seguinte equação:

$média = ((P1 + P2) / 2) * 0.8 + (T * 0.2)$. Seu programa deverá gravar em um outro arquivo: o nome de cada aluno e sua respectiva média (uma linha para cada aluno). Os nomes dos arquivos de entrada e saída devem ser informados pelo usuário. Em seguida, abra o arquivo criado usando um editor de textos qualquer e verifique o resultado do programa.

Exercício 6

- Escreva um programa que calcule e informe o número de vezes que uma determinada palavra aparece em um **arquivo texto**. Para testar seu programa, crie um arquivo em um editor de textos qualquer. Solicite ao usuário o nome do arquivo a ser utilizado e a palavra a ser buscada. A busca deve ser case insensitive (não há diferenciação entre maiúsculas e minúsculas).
- Para simplificar, considere que o arquivo texto não conterá nenhum tipo de pontuação.

Exercício 7

Implemente as funções usadas no programa principal a seguir:

- **grava(pFile, N):** armazena no arquivo binário (pFile) valores inteiros de 1 até N, em ordem decrescente.
- **recupera(pFile):** retorna um ponteiro para uma região de memória alocada dinamicamente, onde são armazenados todos os dados recuperados a partir do arquivo (pFile) através de uma única chamada da função fread (todos os N números armazenados no arquivo).
- **exibe_vetor(vetor, N):** exibe em tela todos os N elementos armazenados em vetor.

Exercício 7

- **modifica(pFile,M)**: verifica os últimos M números armazenados no arquivo (pFile) e dentre estes, substituiu (neste arquivo) os números que forem pares pelos seus dobros. O intervalo permitido para M é $(0, N]$.
- **recupera_exibe(pFile)**: deve recuperar todos os números gravados no arquivo (pFile) e exibí-los em tela.

No caso das funções tipadas, não esqueça de **verificar os resultados de retorno que indicam erros**, conforme pode ser observado a partir dos testes presentes no programa principal.

Exercício 7

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *pFile;
    int i, N, M, num, mod, *vetor;
    char nome_arquivo[100];
    printf("Informe o nome do arquivo a ser criado: ");
    gets(nome_arquivo);
    printf("Informe um numero positivo: ");
    scanf("%d", &N);
    pFile = fopen (nome_arquivo , "wb+");
    if (!pFile){
        printf("Erro na criacao do arquivo...");
        return -1;
    }
    if (grava(pFile, N) != N){
        printf("Erro na gravacao do arquivo...");
        return -1;
    }
    vetor=recupera(pFile);
    if(vetor==NULL){
        printf("Erro na recuperacao de dados do arquivo...");
        return -1;
    }
    exibe_vetor(vetor,N);

    printf("Informe um numero positivo M correspondente a quantidade de valores a
partir do final do arquivo que deseja alterar: ");
    scanf("%d", &M);
    mod=modifica(pFile,M);
    if(mod==-1){
        printf("Valor de M fora do intervalo permitido...");
        return -1;
    }
    if(mod!=M){
        printf("Erro de leitura e/ou gravacao no arquivo...");
        return -1;
    }
    recupera_exibe(pFile);
    fclose (pFile);
    return 0;
}
```

Exercício 7

Exemplo de entrada/saída em tela durante a execução do programa:

```
Informe o nome do arquivo a ser criado: teste.dat
```

```
Informe um numero positivo: 20
```

```
20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
```

```
Informe um numero positivo M correspondente a quantidade  
de valores a partir do final do arquivo que deseja  
alterar: 10
```

```
20 19 18 17 16 15 14 13 12 11 20 9 16 7 12 5 8 3 4 1
```

Exercício 8

Escreva um programa que crie um arquivo binário chamado "**alunos.dat**" para gravar os dados sobre alunos de uma turma. Devem ser solicitados ao usuário, sobre cada aluno, o nome e as notas em três avaliações. A média aritmética das 3 notas (**P1**, **P2** e **P3**) deve ser calculada pelo programa para cada um dos alunos. Esses dados devem ser armazenados no arquivo binário, utilizando a estrutura **Taluno** abaixo:

```
typedef struct
{
    char nome[100];
    double nota[3];
    double media;
} Taluno;
```

O programa deve terminar a solicitação dos dados caso seja informado um nome vazio (apenas <Enter>).

Exercício 9

Faça um outro programa que leia todos os dados armazenados no arquivo binário "**alunos.dat**" (conforme gravado no exercício anterior) e apresente-os em tela. O programa deve atualizar o arquivo, modificando o valor da média pelo resultado obtido através do cálculo de média ponderada a seguir:

$$0.25 * P1 + 0.35 * P2 + 0.4 * P3$$

Além disso, seu programa deverá calcular e exibir o percentual de alunos que tiveram a média atualizada devido à modificação feita no cálculo da média (pelo uso de média ponderada em vez de média aritmética).

Após a atualização do arquivo, o programa deverá ler e imprimir na tela todos os dados (nomes, notas e médias) dos alunos armazenados no arquivo (já com as médias atualizadas).

Desafio 5

Implemente as funções sendo usadas no programa principal a seguir:

grava(pFile, N): armazena no arquivo binário (pFile) informações (Tpessoa) de N pessoas, na mesma ordem de entrada.

modifica(pFile, M, idade): substitui a idade da M-ésima pessoa do arquivo binário (pFile) pelo valor de idade. A função deve verificar se M está dentro do intervalo inteiro permitido (de 1 até a quantidade de registros existentes no arquivo).

exibe(pFile): deve recuperar todas as informações das pessoas (TPessoa) gravadas no arquivo binário (pFile) e exibí-las em tela.

Desafio 5

```
#include <string.h>
#include <stdlib.h>
#include <stdio.h>

typedef struct {
    char nome[30];
    int idade;
} Tpessoa;

int main()
{
    FILE *pFile;
    int N, M, mod, idade;
    char nome_arquivo[101];
    fgets(nome_arquivo, 100, stdin); //nome do arquivo a ser criado
    nome_arquivo[strlen(nome_arquivo)-1]='\0';
    scanf("%d", &N); //numero de pessoas desejado

    pFile = fopen (nome_arquivo , "wb+");

    if (!pFile){
        printf("Erro na criacao do arquivo...");
        return -1;
    }

    if (grava(pFile, N) != N){
        printf("Erro na gravacao do arquivo...");
        return -1;
    }
    scanf("%d", &M); //numero correspondente a ordem de entrada da pessoa que se deseja alterar a idade
    scanf("%d", &idade); //valor da idade a ser atualizado para a M-ésima pessoa
    mod=modifica(pFile, M, idade);
    if(mod==-1){
        printf("Valor de M fora do intervalo permitido...");
        return -1;
    }
    if(mod==0){
        printf("Erro de leitura e/ou gravacao no arquivo...");
        return -1;
    }

    exibe(pFile);

    fclose (pFile);
    return 0;}
```

Desafio 5

- O programa principal deve funcionar sem nenhuma alteração além da implementação das funções solicitadas.
- No caso das funções tipadas (**grava** e **modifica**), não esqueça de verificar os resultados de retorno que indicam erros, conforme pode ser observado a partir dos testes presentes no programa principal.

Desafio 5

- Exemplos de entradas e saídas em tela

Exemplo 1

Entrada:	Saída:
saida1.dat	Mario - 20 anos
5	Ana - 19 anos
Mario 20	Denis - 45 anos
Ana 19	Julia - 43 anos
Denis 50	Luis - 32 anos
Julia 43	
Luis 32	
3	
45	

Exemplo 2

Entrada:	Saída:
saida2.bin	Leandro - 11 anos
1	
Leandro 25	
1	
11	

- Exemplos de entradas e saídas em tela

Exemplo 3

Entrada:	Saída:
saida3	Valor de M fora
2	do intervalo
Daniel 32	permitido...
Roberto 40	
0	
11	

Exemplo 4

Entrada:	Saída:
saida4	Valor de M fora
2	do intervalo
Daniel 32	permitido...
Roberto 40	
3	
20	